

TRAIN COACH MANAGEMENT SYSTEM

Using Singly Linked List

1. Problem Statement

Managing railway coaches becomes difficult when coaches need to be added, removed, searched, or displayed dynamically. A fixed-size array is not ideal because the number of coaches may change during operation. The Train Coach Management System provides a simple way to manage coaches using a Singly Linked List. Each coach is stored as a node containing the coach number, coach type, and a pointer to the next coach.

The system supports adding a coach, removing a coach, searching for a coach, displaying all coaches, and exiting the program. Dynamic memory allocation is used so that memory is created when a coach is added and released when a coach is deleted.

2. Objectives

- To implement railway coach management using a Singly Linked List.
- To dynamically add new railway coaches without using a fixed-size array.
- To remove a coach using its coach number.
- To search for a particular coach by traversing the linked list.
- To display all available coaches and their details in sequence.
- To understand dynamic memory allocation using new and delete.
- To demonstrate practical applications of insertion, deletion, searching, and traversal.

3. Algorithms

Algorithm 1: Add Coach

1. Start.
2. Create a new coach node.
3. Enter coach number and coach type.
4. Set next = NULL.
5. If head == NULL, make the new node head.
6. Otherwise, traverse to the last node and link the new node.
7. Display "Coach added successfully".
8. Stop.

Algorithm 2: Remove Coach

1. Start.
2. If head == NULL, display "No coaches available" and stop.
3. Enter the coach number to remove.
4. If the first node matches, update head and delete the node.
5. Otherwise, search for the required coach.
6. If found, change the links and delete the node.
7. If not found, display "Coach not found".
8. Stop.

Algorithm 3: Display Coaches

1. Start.
2. If head == NULL, display “No coaches available” and stop.
3. Set temp = head.
4. Traverse the linked list.
5. Display coach number and coach type of each node.
6. Stop.

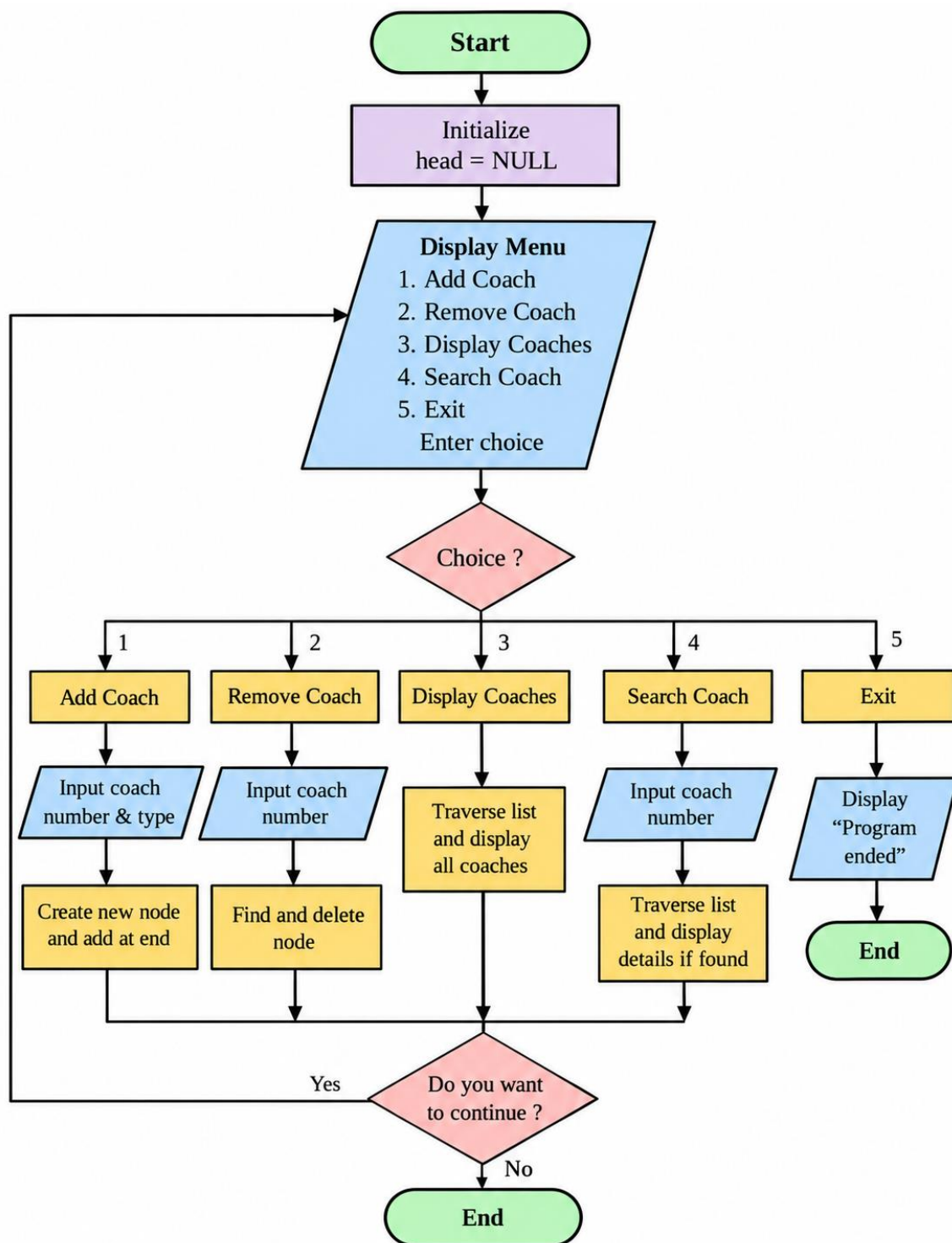
Algorithm 4: Search Coach

1. Start.
2. If head == NULL, display “No coaches available” and stop.
3. Enter the coach number to search.
4. Set temp = head.
5. Compare the coach number with each node.
6. If found, display coach details.
7. If temp == NULL, display “Coach not found”.
8. Stop.

Algorithm 5: Main Program

1. Start.
2. Initialize head = NULL.
3. Display the menu.
4. Read the user's choice.
5. Perform Add, Remove, Display, or Search operation.
6. If choice is 5, exit the program.
7. Otherwise, return to the menu.
8. Stop.

4. Flowchart



5. Code

```
#include <iostream>
using namespace std;

struct Coach
{
    int coachNumber;
    string coachType;
    Coach *next;
};

Coach *head = NULL;

// Add a new coach
void addCoach()
{
    Coach *newCoach = new Coach;

    cout << "\nEnter Coach Number: ";
    cin >> newCoach->coachNumber;

    cout << "Enter Coach Type (AC/SL/GEN): ";
    cin >> newCoach->coachType;

    newCoach->next = NULL;

    if (head == NULL)
    {
        head = newCoach;
    }
    else
    {
        Coach *temp = head;

        while (temp->next != NULL)
        {
            temp = temp->next;
        }

        temp->next = newCoach;
    }

    cout << "\nCoach added successfully!\n";
}

// Remove a coach
void removeCoach()
{

```

```

int number;

if (head == NULL)
{
    cout << "\nNo coaches available!\n";
    return;
}

cout << "\nEnter Coach Number to remove: ";
cin >> number;

// Delete first node
if (head->coachNumber == number)
{
    Coach *temp = head;
    head = head->next;
    delete temp;

    cout << "\nCoach removed successfully!\n";
    return;
}

Coach *temp = head;

while (temp->next != NULL &&
    temp->next->coachNumber != number)
{
    temp = temp->next;
}

if (temp->next == NULL)
{
    cout << "\nCoach not found!\n";
    return;
}

Coach *deleteNode = temp->next;
temp->next = temp->next->next;

delete deleteNode;

cout << "\nCoach removed successfully!\n";
}

// Display all coaches
void displayCoaches()
{
    if (head == NULL)
    {

```

```

        cout << "\nNo coaches available!\n";
        return;
    }

    Coach *temp = head;

    cout << "\n===== TRAIN COACHES =====\n";

    while (temp != NULL)
    {
        cout << "Coach Number : " << temp->coachNumber << endl;
        cout << "Coach Type   : " << temp->coachType << endl;
        cout << "-----\n";

        temp = temp->next;
    }
}

// Search for a coach
void searchCoach()
{
    int number;

    if (head == NULL)
    {
        cout << "\nNo coaches available!\n";
        return;
    }

    cout << "\nEnter Coach Number to search: ";
    cin >> number;

    Coach *temp = head;

    while (temp != NULL)
    {
        if (temp->coachNumber == number)
        {
            cout << "\nCoach Found!\n";
            cout << "Coach Number : " << temp->coachNumber << endl;
            cout << "Coach Type   : " << temp->coachType << endl;
            return;
        }

        temp = temp->next;
    }

    cout << "\nCoach not found!\n";
}

```

```

// Main function
int main()
{
    int choice;

    do
    {
        cout << "\n===== TRAIN COACH MANAGEMENT SYSTEM =====";
        cout << "\n1. Add Coach";
        cout << "\n2. Remove Coach";
        cout << "\n3. Display Coaches";
        cout << "\n4. Search Coach";
        cout << "\n5. Exit";

        cout << "\n\nEnter your choice: ";
        cin >> choice;

        switch (choice)
        {
            case 1:
                addCoach();
                break;

            case 2:
                removeCoach();
                break;

            case 3:
                displayCoaches();
                break;

            case 4:
                searchCoach();
                break;

            case 5:
                cout << "\nProgram ended.\n";
                break;

            default:
                cout << "\nInvalid choice!\n";
        }

    } while (choice != 5);

    return 0;
}

```

5. Output

```
===== TRAIN COACH MANAGEMENT SYSTEM =====
1. Add Coach
2. Remove Coach
3. Display Coaches
4. Search Coach
5. Exit

Enter your choice: 1

Enter Coach Number: 101
Enter Coach Type (AC/SL/GEN): AC

Coach added successfully!

===== TRAIN COACH MANAGEMENT SYSTEM =====
1. Add Coach
2. Remove Coach
3. Display Coaches
4. Search Coach
5. Exit

Enter your choice: 1

Enter Coach Number: 102
Enter Coach Type (AC/SL/GEN): SL

Coach added successfully!

===== TRAIN COACH MANAGEMENT SYSTEM =====
1. Add Coach
2. Remove Coach
3. Display Coaches
4. Search Coach
5. Exit

Enter your choice: 1

Enter Coach Number: 103
Enter Coach Type (AC/SL/GEN): GEN

Coach added successfully!

===== TRAIN COACH MANAGEMENT SYSTEM =====
1. Add Coach
2. Remove Coach
3. Display Coaches
4. Search Coach
5. Exit

Enter your choice: 3

===== TRAIN COACHES =====
Coach Number : 101
Coach Type   : AC
-----
Coach Number : 102
Coach Type   : SL
-----
Coach Number : 103
Coach Type   : GEN
-----

===== TRAIN COACH MANAGEMENT SYSTEM =====
1. Add Coach
2. Remove Coach
3. Display Coaches
4. Search Coach
5. Exit

Enter your choice: |
```